ConstructFlow System – Structural Design Patterns Technical Report

---

1. Introduction

ConstructFlow is a construction management system designed to handle complex operations such as project management, resource allocation, reporting, and notifications. To ensure scalability, maintainability, and flexibility, several **structural design patterns** were implemented.

This report presents a detailed analysis of the implemented patterns, including:

- Problem identification
- Pattern selection justification
- UML class diagrams (provided)
- Implementation explanation

The patterns implemented are:

- Bridge (2 cases)
- Adapter (2 cases)
- Flyweight (1 case)
- Decorator (2 cases)

---

2. Bridge Pattern – Notification System

Problem

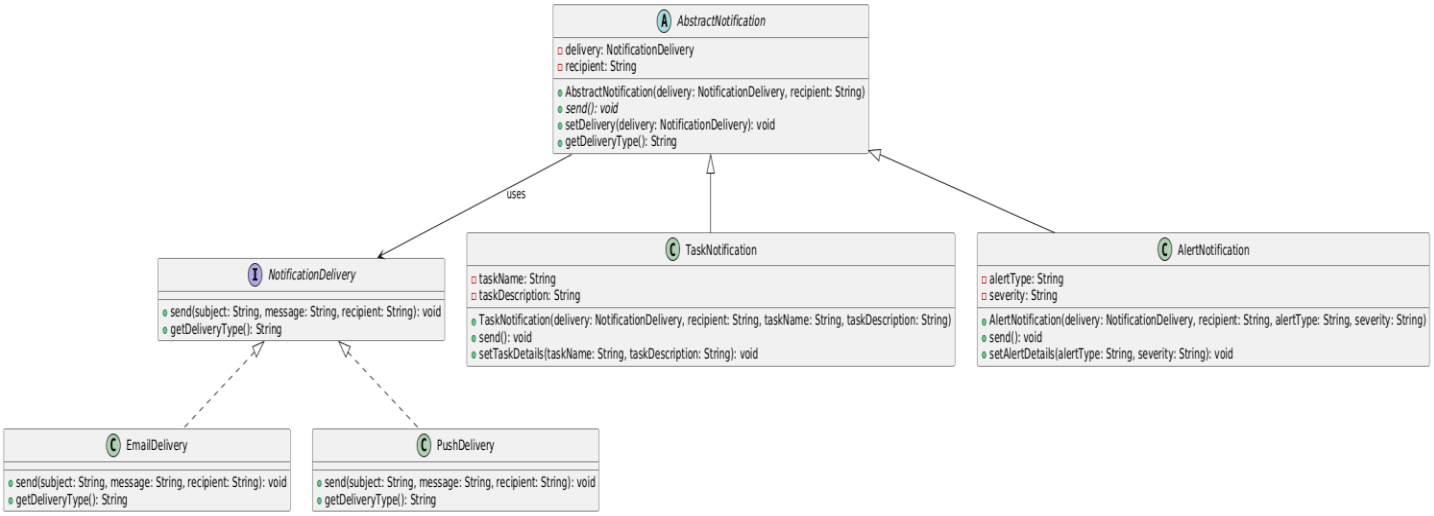
The system must support multiple notification types (e.g., task notifications, alerts) and multiple delivery channels (email, push). Without proper design, this leads to class explosion such as:

- EmailTaskNotification
- PushTaskNotification
- EmailAlertNotificatio

Solution

The **Bridge pattern** separates:

- Abstraction → Notification types
- Implementation → Delivery mechanisms



How It Works

- `AbstractNotification` holds a reference to `NotificationDelivery`
- Concrete notifications (`TaskNotification`, `AlertNotification`) delegate sending to delivery objects
- Delivery types (`EmailDelivery`, `PushDelivery`) implement sending logic

Why Bridge

- Avoids combinational class explosion
- Allows independent extension of:
  - notification types
  - delivery methods

Result

- Runtime switching of delivery channels
- Clean separation of responsibilities

3. Bridge Pattern – Database System

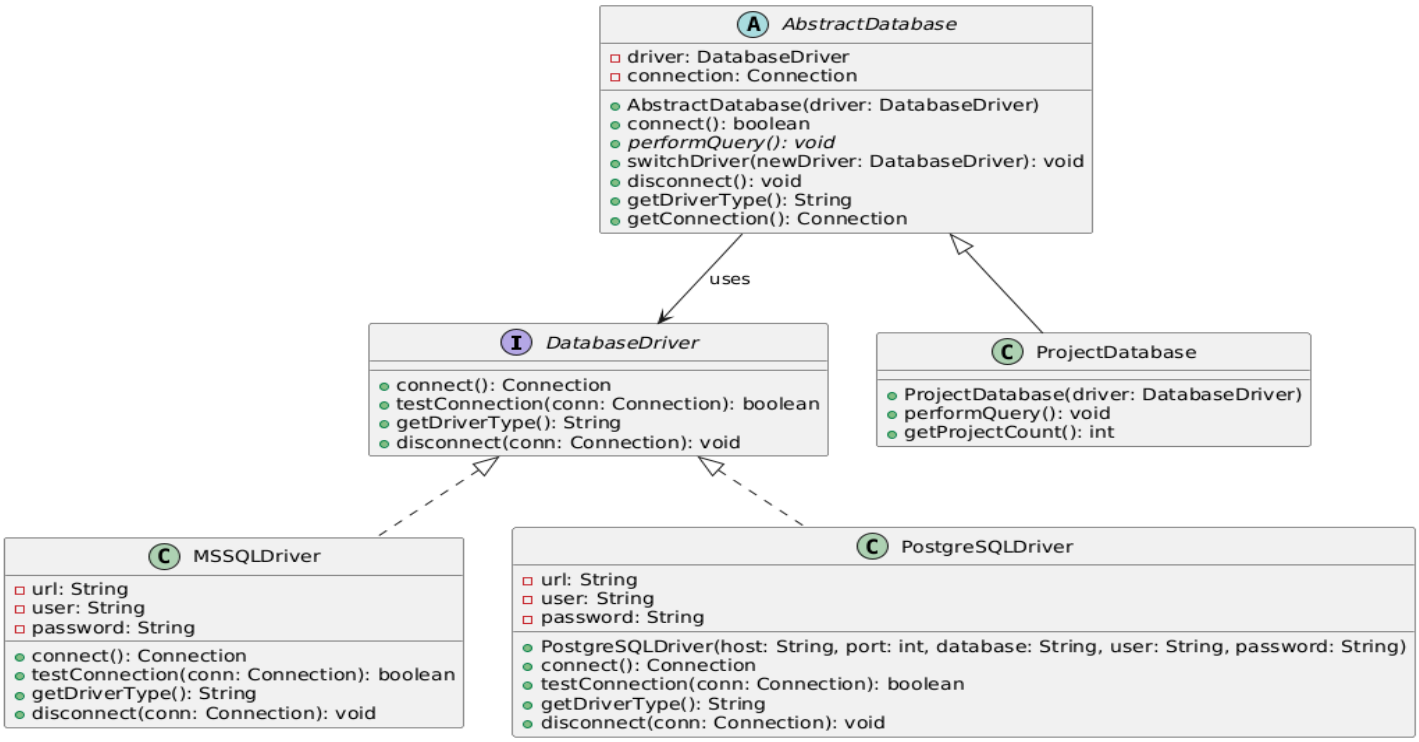
Problem

The system must support multiple databases (MSSQL, PostgreSQL) without rewriting business logic.

Solution

Bridge separates:

- Abstraction → database operations
- Implementation → database drivers



How It Works

- AbstractDatabase uses a DatabaseDriver
- ProjectDatabase defines operations
- Drivers (MSSQLDriver, PostgreSQLDriver) implement connection logic

Why Bridge

- Enables database independence
- Allows switching database drivers at runtime

Result

- Flexible and scalable data layer
- Easy integration of new database systems

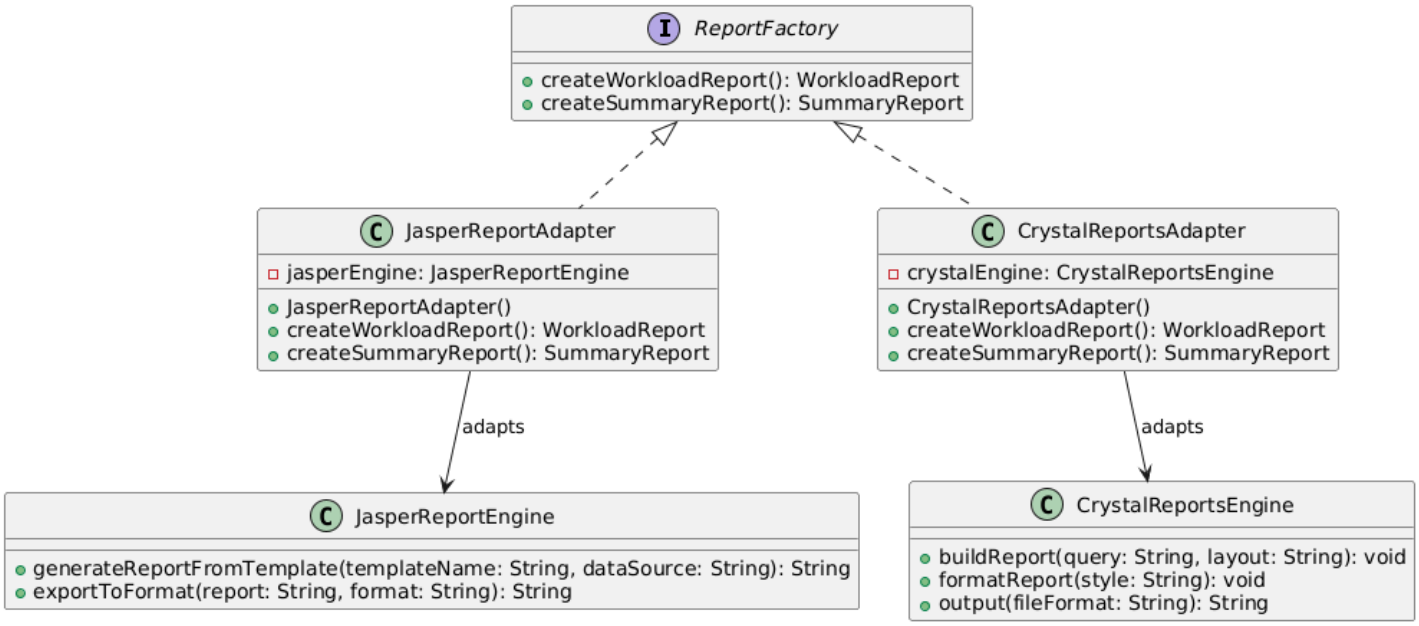
4. Adapter Pattern – Report Engine Integration

Problem

External report engines (Jasper, Crystal Reports) have incompatible APIs.

Solution

Adapter converts each engine's interface into a common interface (`ReportFactory`).



How It Works

- JasperReportAdapter wraps JasperReportEngine
- CrystalReportsAdapter wraps CrystalReportsEngine
- Both implement a unified interface

Why Adapter

- Allows reuse of third-party libraries without modification
- Ensures consistent system interface

Result

- Seamless integration of different reporting tools
- Easy replacement or extension

---

5. Adapter Pattern – Data Import System

Problem

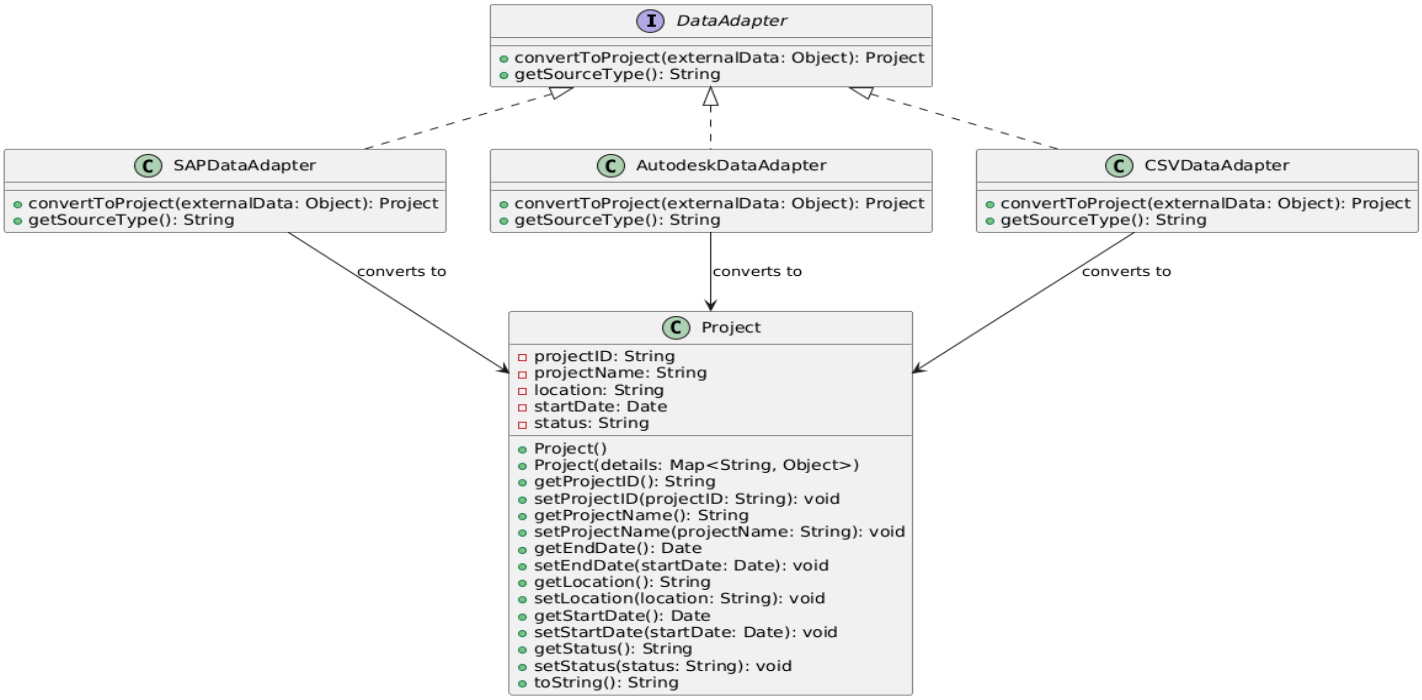
The system must import data from:

- SAP
- Autodesk
- CSV

Each uses different formats.

Solution

Adapter standardizes data into a unified `Project` object.



How It Works

- `DataAdapter` defines conversion interface
- Each adapter converts external data → internal model

Why Adapter

- Decouples external systems from internal logic
- Simplifies integration of new data sources

Result

- Clean data transformation layer
- Improved system interoperability

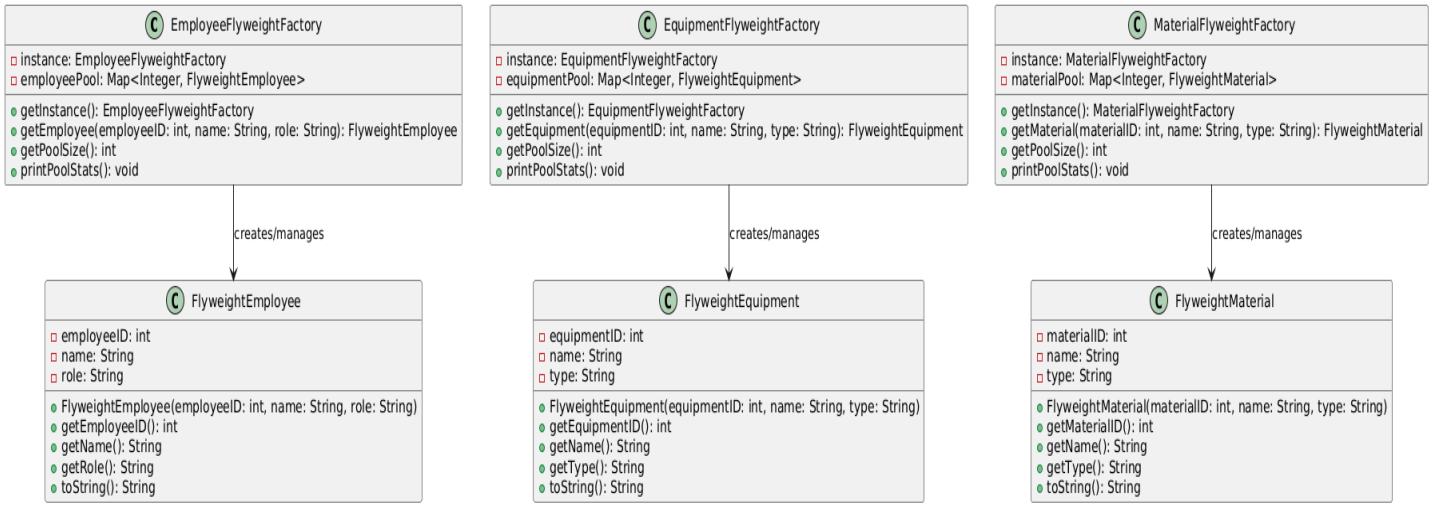
6. Flyweight Pattern – Resource Management

Problem

Repeated creation of identical objects (employees, materials, equipment) wastes memory.

Solution

Flyweight shares objects using a central factory.



How It Works

- Factories store objects in a map (pool)
- If object exists → reuse
- If not → create and store

Why Flyweight

- Reduces memory usage
- Avoids duplicate objects

Result

- Efficient resource handling
- Improved performance in large-scale systems

---

7. Decorator Pattern – Task System

Problem

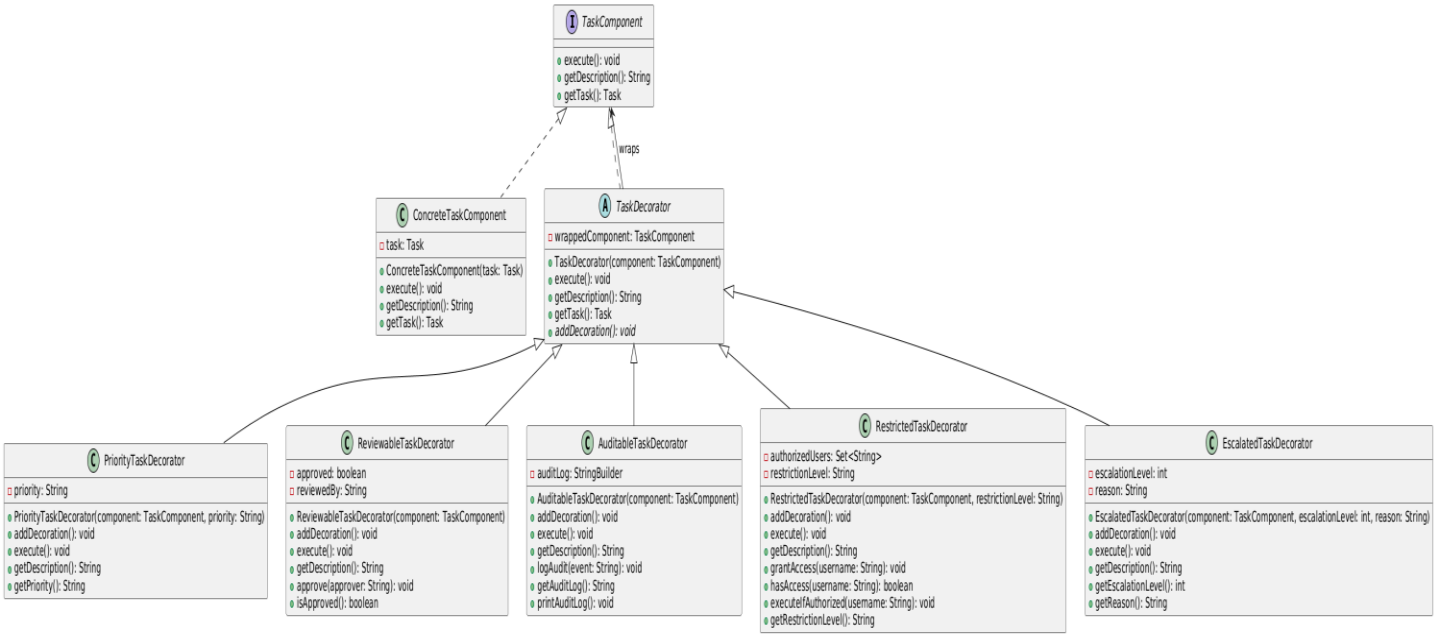
Tasks need dynamic features:

- Priority
- Audit logs
- Restrictions
- Escalation

Inheritance would create too many classes.

Solution

Decorator wraps tasks with additional behavior dynamically.



How It Works

- `TaskComponent` → base interface
- `TaskDecorator` → wraps component
- Concrete decorators add behavior

Why Decorator

- Adds features at runtime
- Avoids subclass explosion

Result

- Flexible task enhancement system
- Clean and modular code

8. Decorator Pattern – Report Enhancement

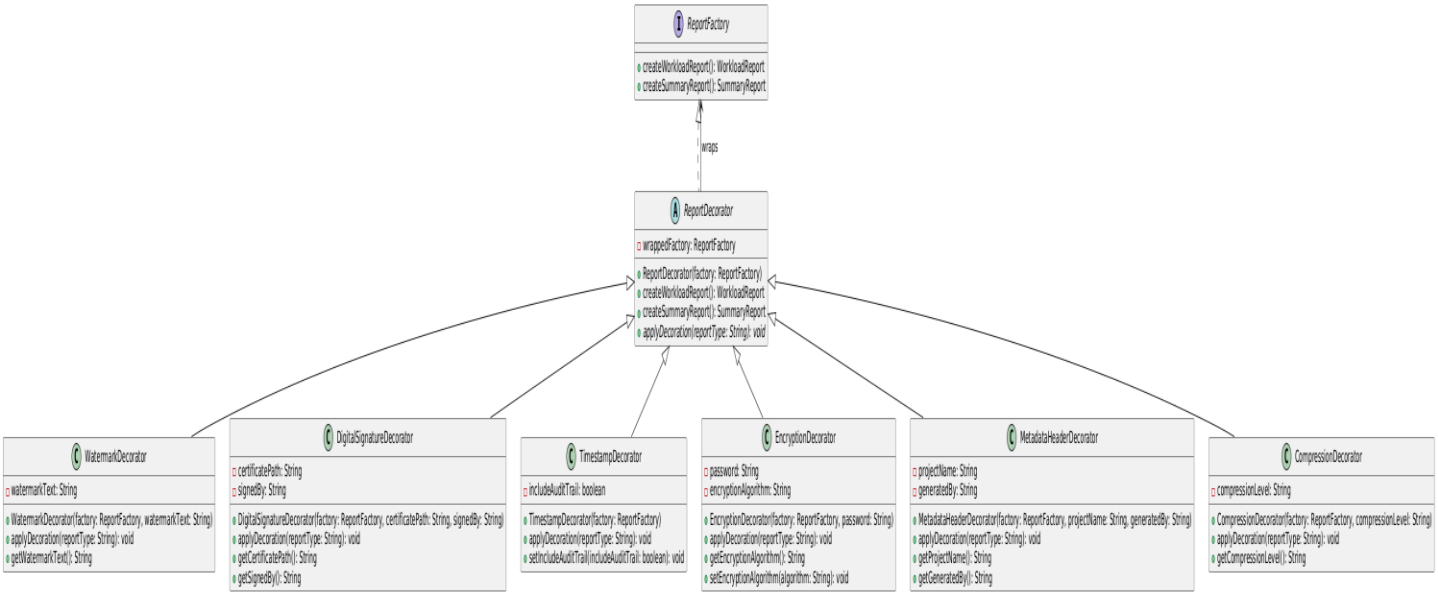
Problem

Reports require multiple optional features:

- Watermark
- Encryption
- Signature
- Metadata

Solution

Decorator allows combining features dynamically.



How It Works

- Base report is wrapped with decorators
- Each decorator adds functionality

Why Decorator

- Supports multiple feature combinations
- Keeps report generation clean

Result

- Highly customizable reporting system
- Modular enhancement pipeline

9. Overall Benefits

Architecture

- Loose coupling
- High extensibility
- Clear separation of concerns

Performance

- Reduced memory usage (Flyweight)
- Efficient runtime behavior (Decorator)

Maintainability

- Modular design
- Easy debugging and testing

---

10. Conclusion

The implementation of structural design patterns in ConstructFlow significantly improves system flexibility, scalability, and maintainability. Each pattern was carefully selected to address real-world challenges in construction management systems.

The combination of Bridge, Adapter, Flyweight, and Decorator patterns results in:

- A robust and extensible architecture
- Efficient resource management
- Flexible feature composition
- Seamless integration with external systems

This demonstrates strong application of software engineering principles and design pattern best practices.